Chapter 3: CodeQL Database Creation

Rod Trent

Chapter 3: CodeQL Database Creation

Download this chapter as a PDF for offline reading:
Chapter-03-CodeQL-Database-Creation.pdf

In Chapter 2 you created your first CodeQL database. This chapter looks inside that database, explains how extraction differs between interpreted and compiled languages, and shows the commands you use to create, inspect, and manage databases.

Database Contents

A CodeQL database is a directory that holds a complete, queryable snapshot of a codebase for one language at one point in time. It is not a simple copy of the source files. It is a relational representation of the code's structure and meaning.

Inside a typical database directory you will find:

- **Relational data** — tables that capture the abstract syntax tree (AST), control-flow graph, data-flow graph, name binding, types, and other semantic facts. These tables are defined by a language-specific schema (`.dbscheme`).
- **Source archive** — a snapshot of the original source files, used for result highlighting and navigation in the VS Code extension and in SARIF output.
- **Metadata** — information about the language, extraction time, source root, and extractor options.

The schema defines relations such as expressions, statements, methods, and calls. The CodeQL standard libraries then layer object-oriented classes (`Expr`, `Stmt`, `Function`, `Call`, etc.) on top of those raw tables so that most queries never touch the low-level relations directly.

Because the database is a snapshot, queries always see the same consistent view of the code. You can run many different queries against one database without re-extracting the source.

Interpreted vs Compiled Languages

CodeQL uses different extraction strategies depending on the language.

Interpreted languages

JavaScript/TypeScript, Python, and Ruby are extracted by reading the source files directly. The extractor resolves imports and dependencies as needed. No separate build step is required.

```
codeql database create my-js-db \
  --language=javascript \
  --source-root=.
```

(You can also use the combined identifier `javascript-typescript`.)

Compiled languages

C/C++, C#, Go, Java/Kotlin, Rust, and Swift normally require observation of a successful build so the extractor can capture the exact compilation units, macros, generated code, and type information the compiler sees.

CodeQL supports three build modes for compiled languages:

Mode	Description	When to use
none	Analyze source without building (supported for C/C++, C#, Java, Rust)	Quick scans, default setup, when a full build is impractical
autobuild	CodeQL detects and runs the build system	Most projects when none is insufficient
manual	You supply an explicit build command	Complex builds, autobuild failures, maximum accuracy

Example with an explicit build:

```
codeql database create my-java-db \
  --language=java \
  --command="mvn -B package -DskipTests" \
  --source-root=.
```

For multi-language repositories use `--db-cluster` so each language receives its own database subdirectory.

Creation Commands and Inspection

Creating a database

The primary command is:

```
codeql database create <database-path> \
  --language=<language> \
  --source-root=<source-dir> \
  [--command=<build-command>] \
  [--build-mode=none|autobuild|manual]
```

The database directory must not already exist. Always ensure the project builds cleanly (for compiled languages) before you ask CodeQL to trace the build.

Inspecting a database

After creation, verify the database:

```
codeql database info <database-path>
```

This shows the language, source root, creation time, and other metadata.

List the files that were extracted:

```
codeql database export <database-path> --output=export.zip
# or simply explore the directory structure
ls -la <database-path>
```

Inside the database you will typically see a `db-<language>` directory (the relational data), a source archive, and a `codeql-database.yml` file.

Common inspection commands

A small helper script is available in the companion repository at `examples/chapter03/inspect-db.sh`:

```
#!/usr/bin/env bash
# Inspect a CodeQL database
set -e
DB="${1:-my-first-db}"

echo "=== Database info ==="
codeql database info "$DB"

echo ""
echo "=== Top-level contents ==="
ls -la "$DB"

echo ""
echo "=== Language directories ==="
find "$DB" -maxdepth 2 -type d -name "db-*"
```

Downloading pre-built databases

For many public GitHub repositories you can download a ready-made CodeQL database instead of creating one. This is useful when you want to practice queries on large real-world codebases. Look for the Code scanning / CodeQL databases section on the repository page, or use the GitHub REST API.

Next Steps

You now understand what a CodeQL database contains and how it is created for both interpreted and compiled languages. In the next chapter we turn to the QL language itself—predicates, classes, and the declarative model that lets you ask precise questions of the data stored in these databases.

All commands and the inspection script referenced above live in the companion repository: github.com/rod-trent/MustLearnCodeQL under `examples/chapter03/`.
